CS100433
Ray tracing

Junqiao Zhao 赵君峤

Department of Computer Science and Technology

College of Electronics and Information Engineering

Tongji University

RAY TRACING: INTRODUCED BY TURNER WHITTED IN 1979

Graphics and Image Processing
J.D. Foley Editor

An Improved Illumination Model for Shaded Display

Turner Whitted
Bell Laboratories
Holmdel, New Jersey

To accurately render a two-dimensional image of a three-dimensional scene, global illumination information that affects the intensity of each pixel of the image must be known at the time the intensity is calculated. In a simplified form, this information is stored in a tree of "rays" extending from the viewer to the first surface encountered and from there to other surfaces and to the light sources. A visible surface algorithm creates this tree for each pixel of the display and passes it to the shader. The shader then traverses the tree to determine the intensity of the light received by the viewer. Consideration of all of these factors allows the shader to accurately simulate true reflection, shadows, and refraction, as well as the effects simulated by conventional shaders. Anti-aliasing is included as an integral part of the visibility calculations. Surfaces displayed include curved as well as polygonal surfaces.

The role of the illumination model is to determine how much light is reflected to the viewer from a visible point on a surface as a function of light source direction and strength, viewer position, surface orientation, and surface properties. The shading calculations can be performed on three scales: microscopic, local, and global. Although the exact nature of reflection from surfaces is best explained in terms of microscopic interactions between light rays and the surface [3], most shaders produce excellent results using aggregate local surface data. Unfortunately, these models are usually limited in scope, i.e., they look only at light source and surface orientations, while ignoring the overall setting in which the surface is placed. The reason that shaders tend to operate on local data is that traditional visible surface algorithms cannot provide the necessary global data.

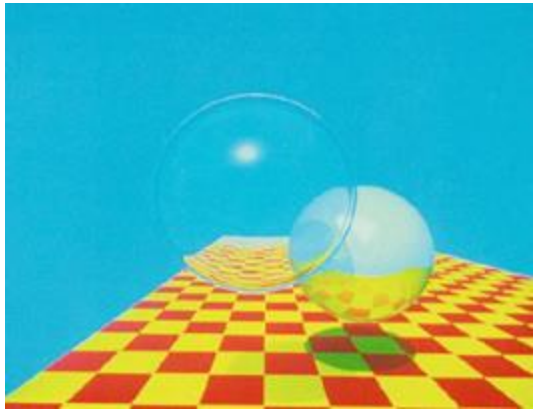
A shading model is presented here that uses global information to calculate intensities. Then, to support this shader, extensions to a ray tracing visible surface algorithm are presented.

1. Conventional Models

The simplest visible surface algorithms use shaders based on Lambert's cosine law. The intensity of the reflected light is proportional to the dot product of the surface normal and the light source direction, simulating a perfect diffuser and yielding a reasonable looking approximation to a dull, matte surface. A more sophisticated model is the one devised by Bui-Tuong Phong [8]. Intensity from Phong's model is given by

$$I = I_a + k_d \sum_{j=1}^{j=ls} (\bar{N} \cdot \bar{L}_j) + k_s \sum_{j=1}^{j=ls} (\bar{N} \cdot \bar{L}'_j)^n, \qquad (1)$$

where

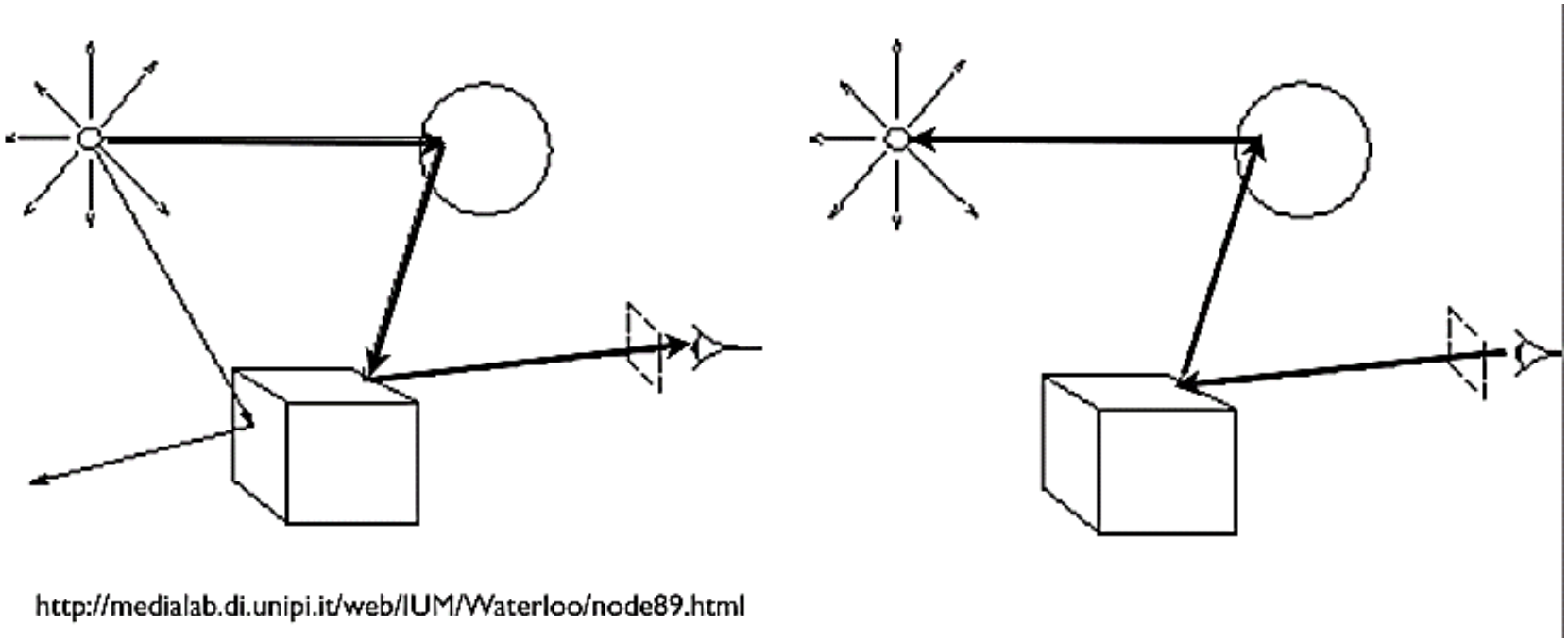


OpenGL vs Ray tracing

- OpenGL is based on a pipeline model in which primitives are rendered one at time
  - No shadows (shadow maps etc.)
  - No multiple reflections (environment maps etc.)
- Global approaches
  - Rendering equation
  - Ray tracing
  - Radiosity
  - Easily making effects: shadows, reflections, transparency

OpenGL vs Ray tracing

- Based on following light rays through the scene
- Iterate over pixels instead of primitives



http://medialab.di.unipi.it/web/IUM/Waterloo/node89.html

Ray tracing

For each pixel {

Shoot a ray from camera to pixel; //Perspective

for all objects in scene

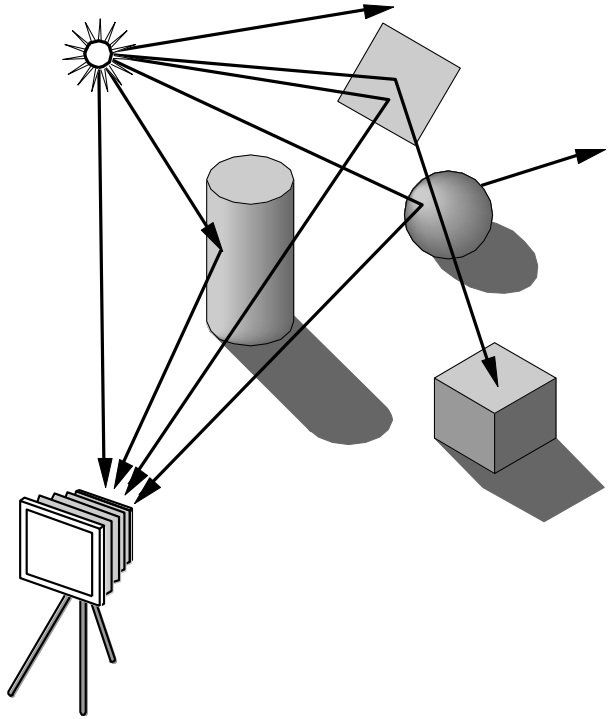
Compute intersection with ray

Find object with closest intersection

Recursively shoot rays from the intersection point

Display color using object + light property

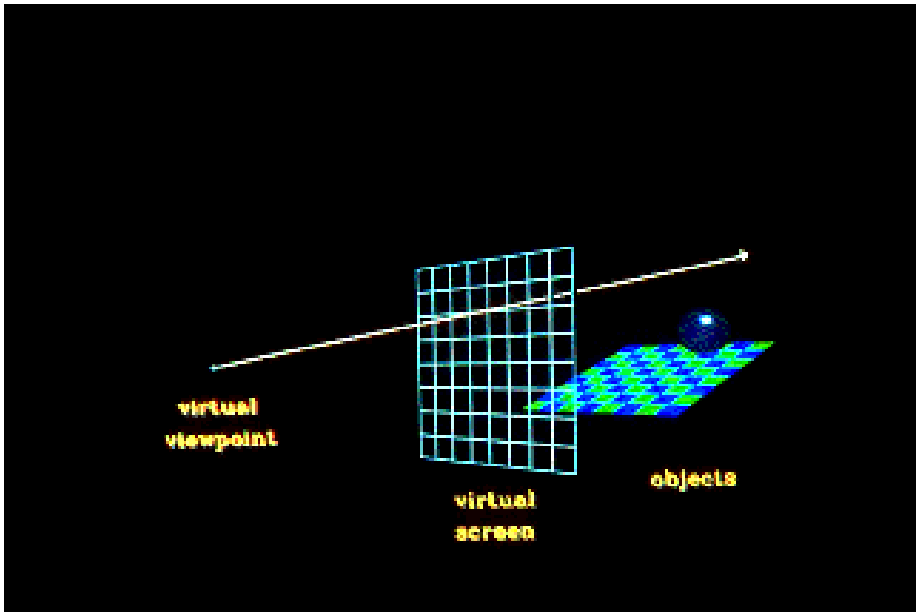
}

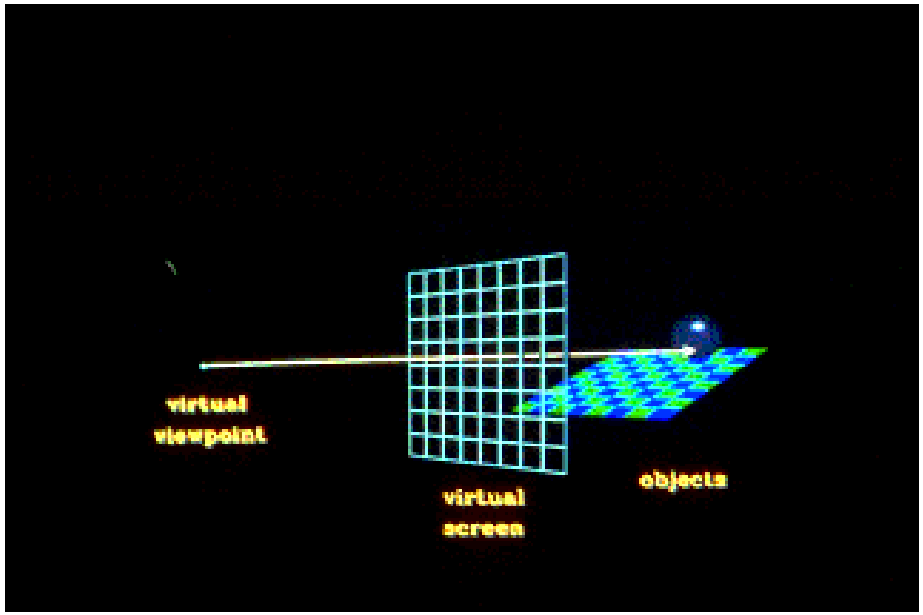


(Ed Angel, NewMexico)

Ray tracing

- Sometimes the **ray misses** all of the objects
- and sometimes the **ray** will **hit** an object

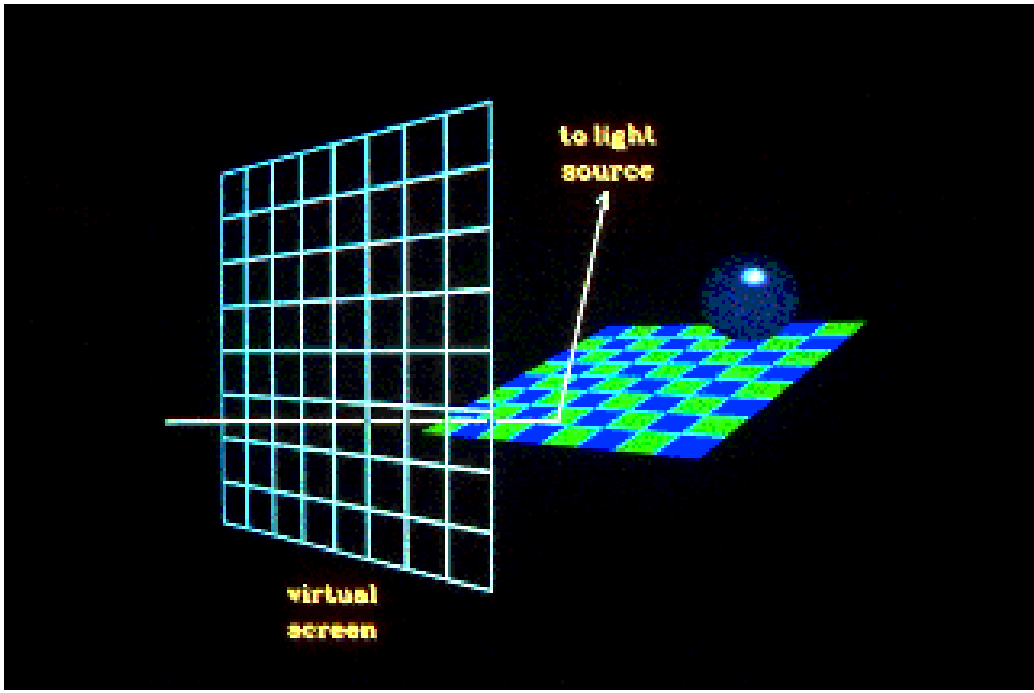




(Siggraph Education: Overview of Ray Tracing)

Ray tracing

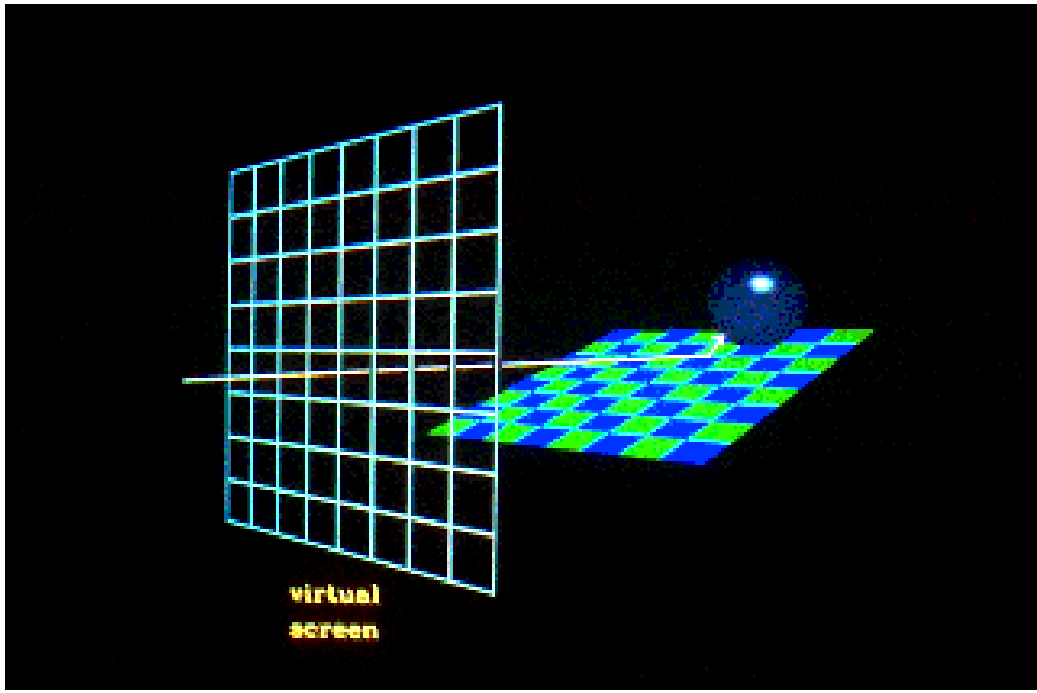
- If the ray hits an object, we want to know if that point on the object is in a shadow.
- So, when the ray hits an object, a secondary ray, called a **"shadow" ray**, is shot towards the light sources.



(Siggraph Education: Overview of Ray Tracing)

Ray tracing

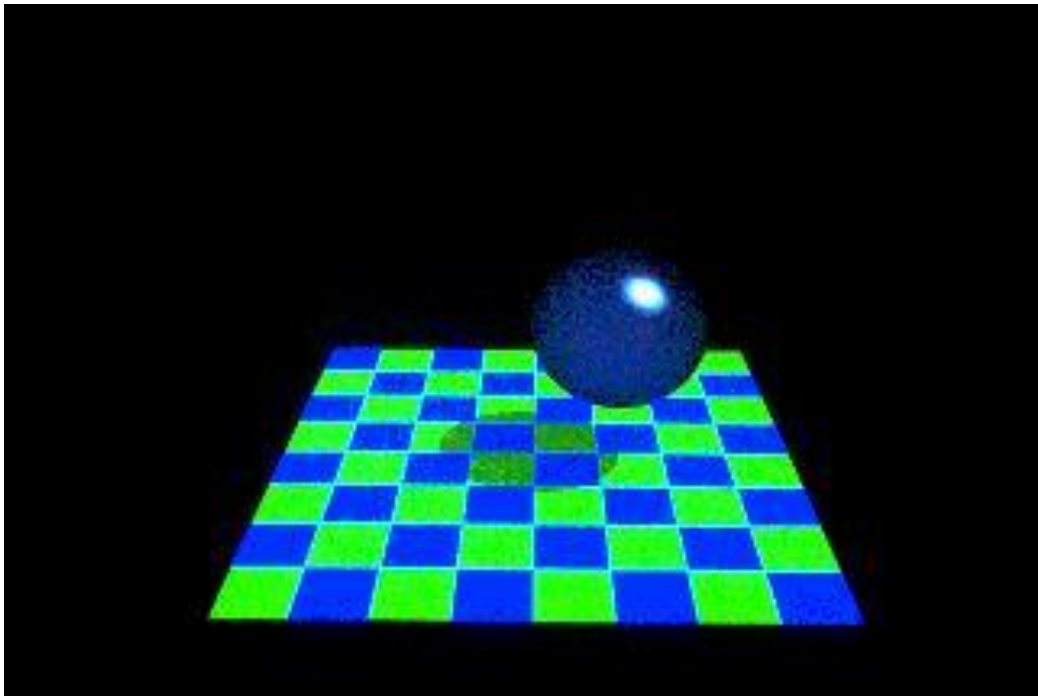
- If this shadow ray hits another object before it hits a light source, then the first intersection point is in the shadow of the second object.
- For a simple illumination model this means that we only apply the ambient term for that light source.



(Siggraph Education: Overview of Ray Tracing)

Ray tracing

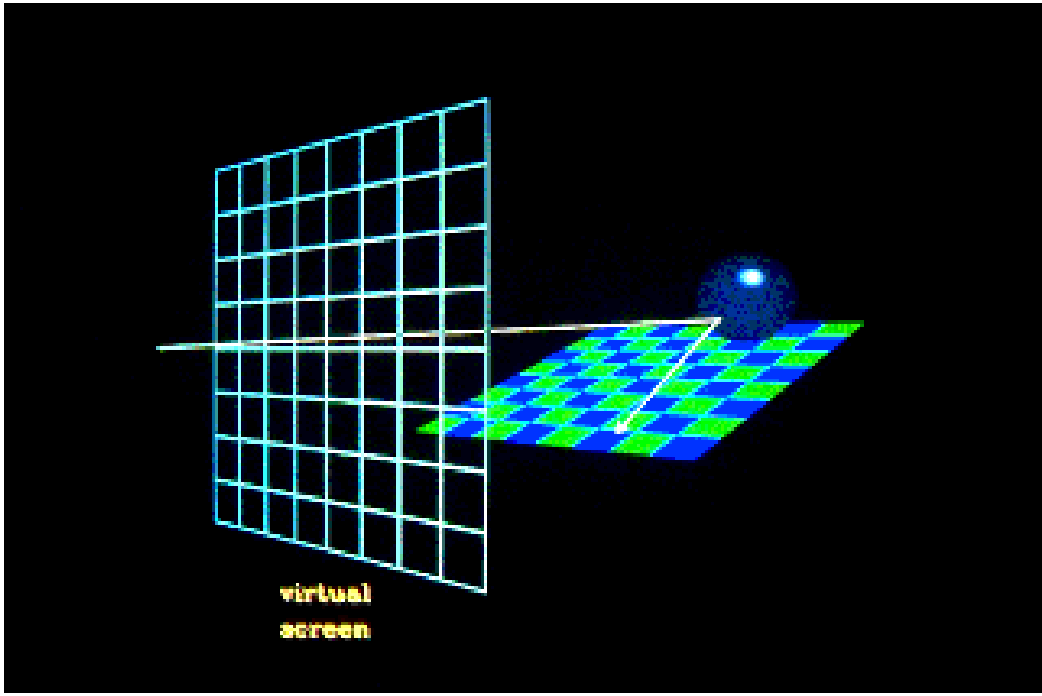
- If this shadow ray hits another object before it hits a light source, then the first intersection point is in the shadow of the second object.
- For a simple illumination model this means that we only apply the ambient term for that light source.



(Siggraph Education: Overview of Ray Tracing)

Ray tracing

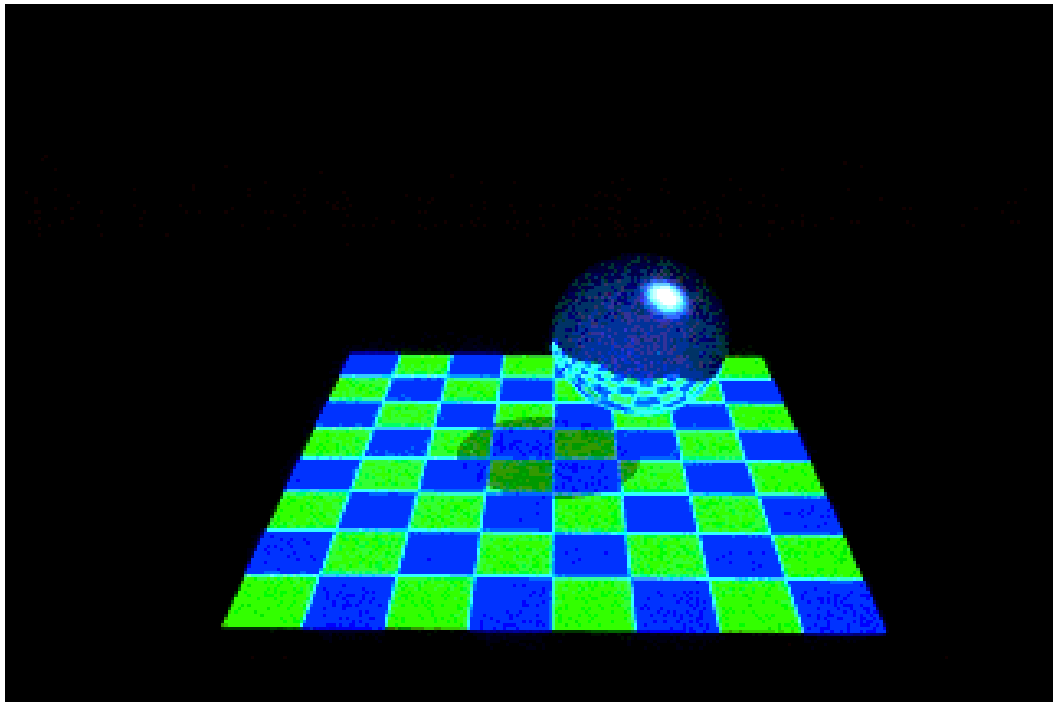
- Also, when a ray hits an object, a **reflected ray** is generated which is tested against all of the objects in the scene.



(Siggraph Education: Overview of Ray Tracing)

Ray tracing

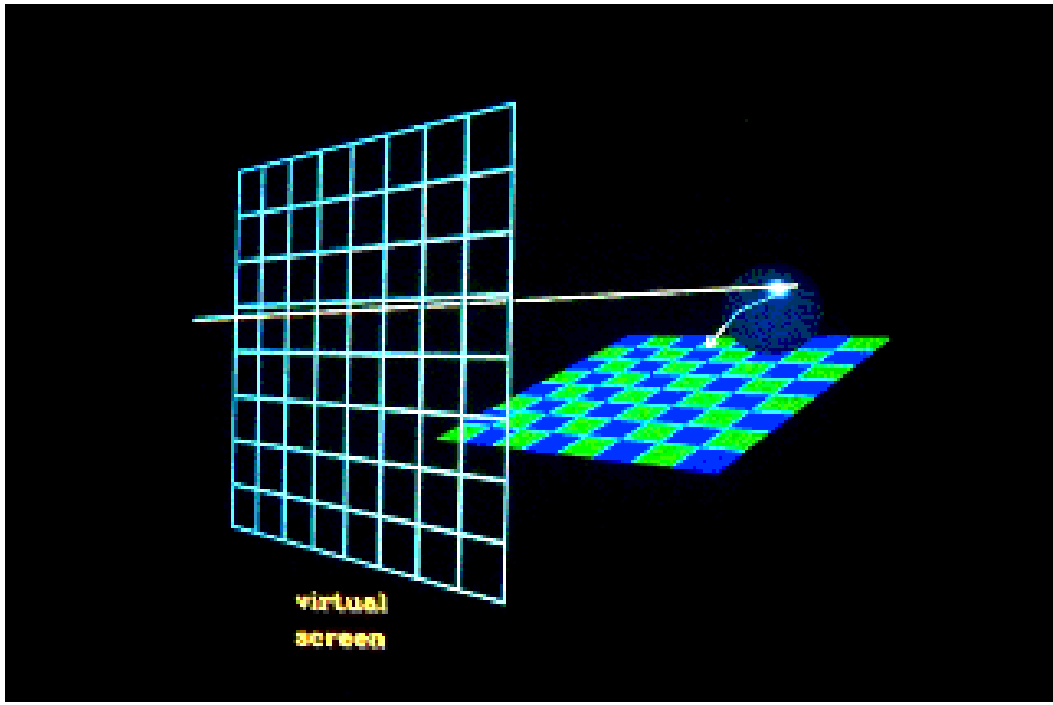
- If the reflected ray hits an object then a local illumination model is applied at the point of intersection and the result is carried back to the first intersection point.



(Siggraph Education: Overview of Ray Tracing)

Ray tracing

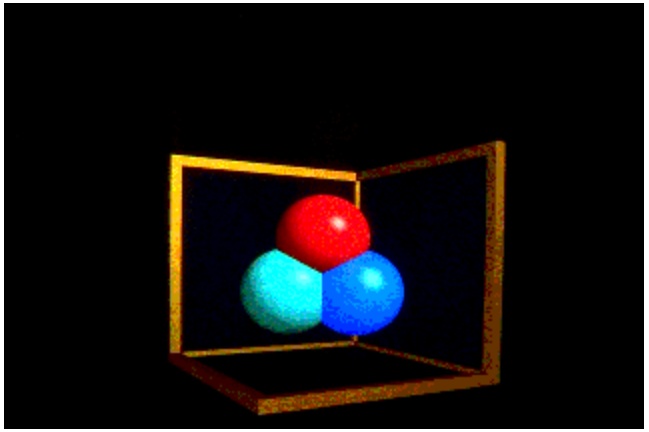
- If the intersected object is transparent, then a **transmitted ray** is generated and tested against all the objects in the scene.

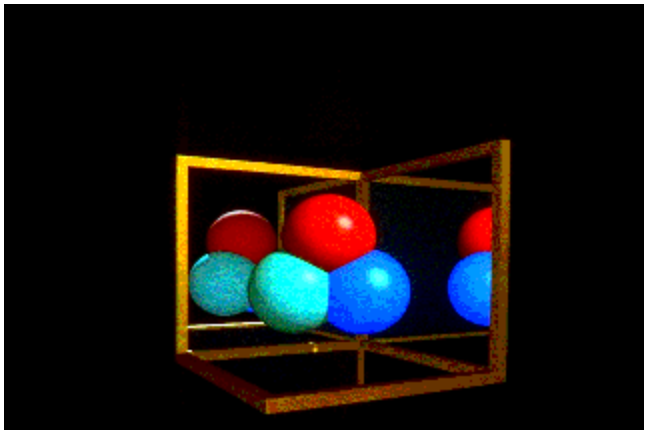


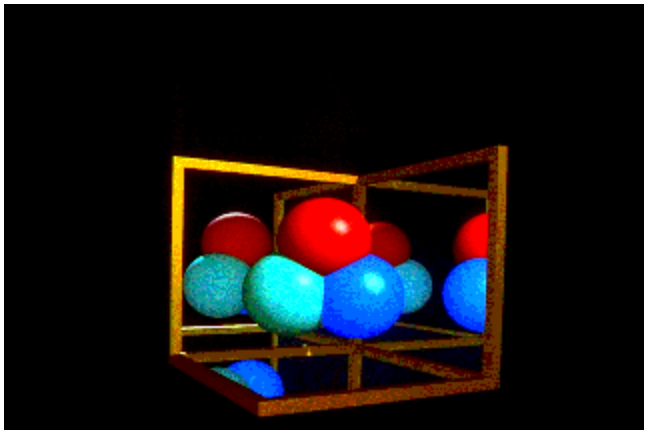
(Siggraph Education: Overview of Ray Tracing)

Ray tracing

- The reflection ray can be implemented recursively
- There can be no reflection or multiple reflection

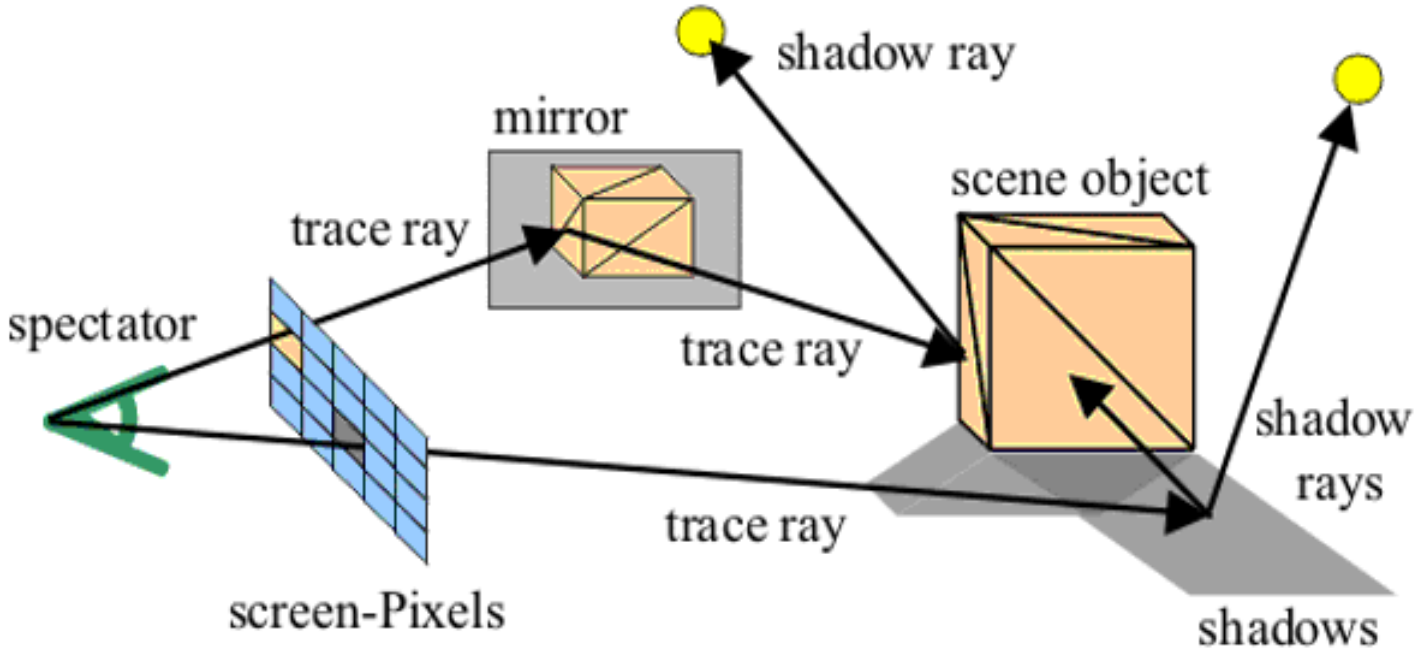






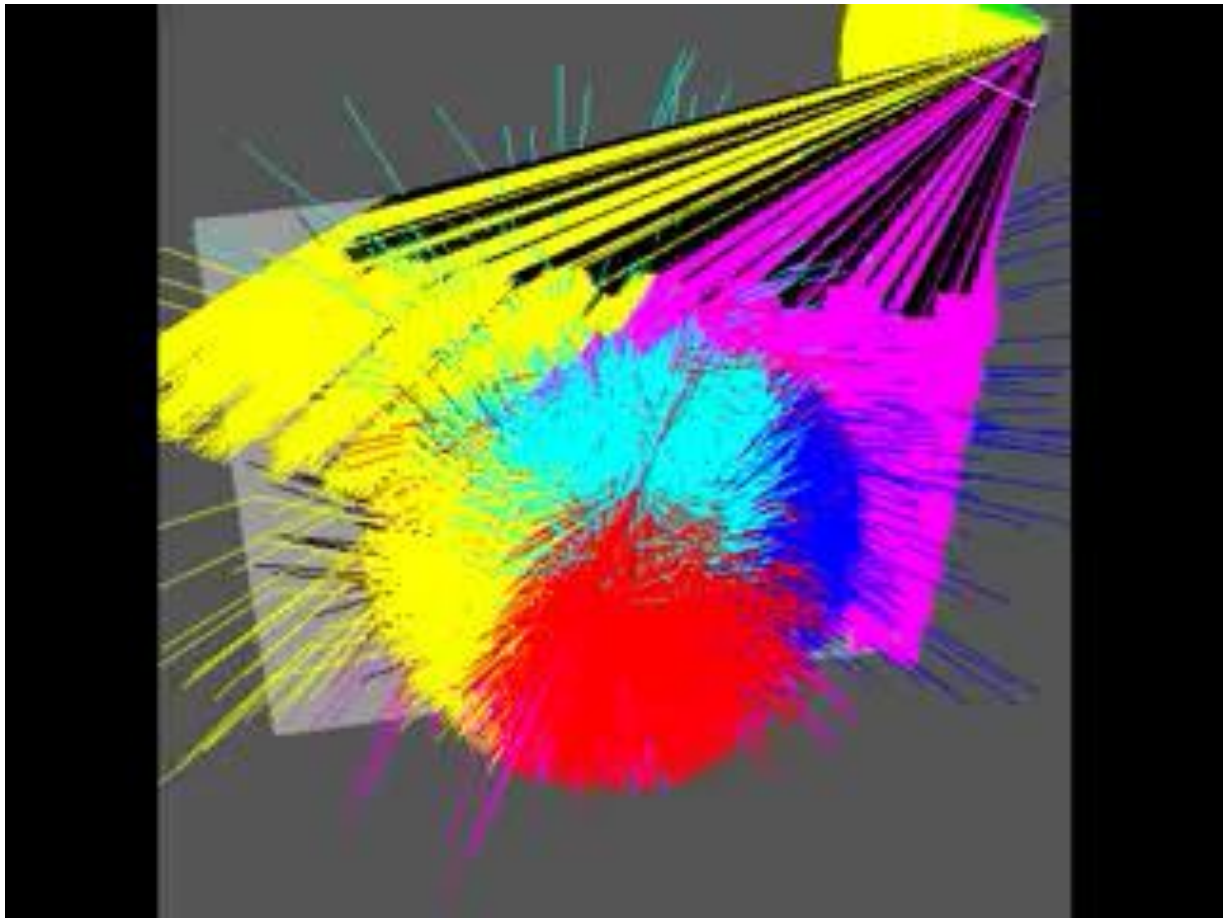
http://www.siggraph.org/education/materials/HyperGraph/raytrace/rtrace0.htm

Rays



(http://www.ice.rwth-aachen.de/research/tools-projects/grace/ray-traycing/)

Visualization of rays



https://www.youtube.com/watch?v=aKqxonOrl4Q

Ray tracing

- Should be able to handle all physical interactions
- Ray tracing paradigm is computation intensive
- Most rays do not affect what we see
- Scattering produces many (infinite) additional rays
- Alternative: ray casting

Diffuse Surfaces

- Theoretically the scattering at each point of intersection generates an infinite number of new rays that should be traced
- We could only trace the transmitted and reflected rays but use the Phong model to compute shade at point of intersection
- We can also sample the scattering rays by using Monto-carlo

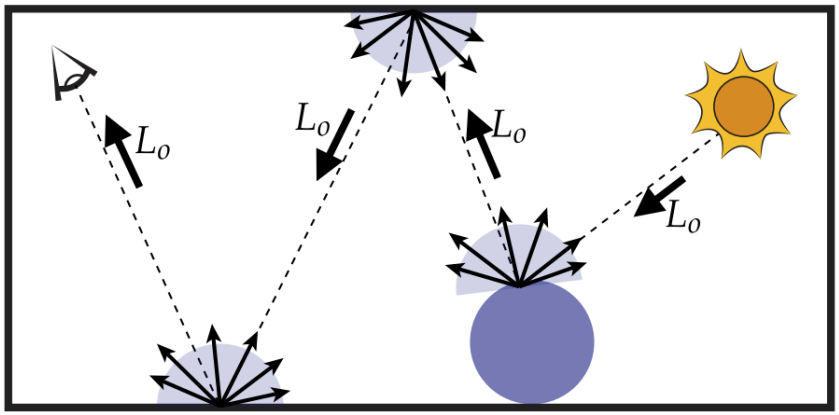
Monte Carlo Ray Tracing

- Apply Monte Carlo integration to the rendering equation
- To determine color of each pixel, integrate incoming light
- What function are we integrating?
  - illumination along different paths of light
  - Each path we trace is a sample
  - How to sample?

$$L_o(p, \omega_o)$$

$$= L_e(p, \omega_o) + \int_{H^2} f_r(p, \omega_i \rightarrow \omega_o) L_i(p, \omega_i) cos\theta d\omega_i$$

$$\lim_{N \to \infty} \frac{|\Omega|}{N} \sum_{i=1}^{N} f(X_i) = \int_{\Omega} f(x)\, dx$$



Direct lighting—uniform sampling (algorithm)

Uniformly-sample hemisphere of directions with respect to solid angle

$$p(\omega) = \frac{1}{2\pi}$$

$$E(\mathrm{p}) = \int L(\mathrm{p}, \omega) \cos\theta \, \mathrm{d}\omega$$

Given surface point p

A ray tracer evaluates radiance along a ray

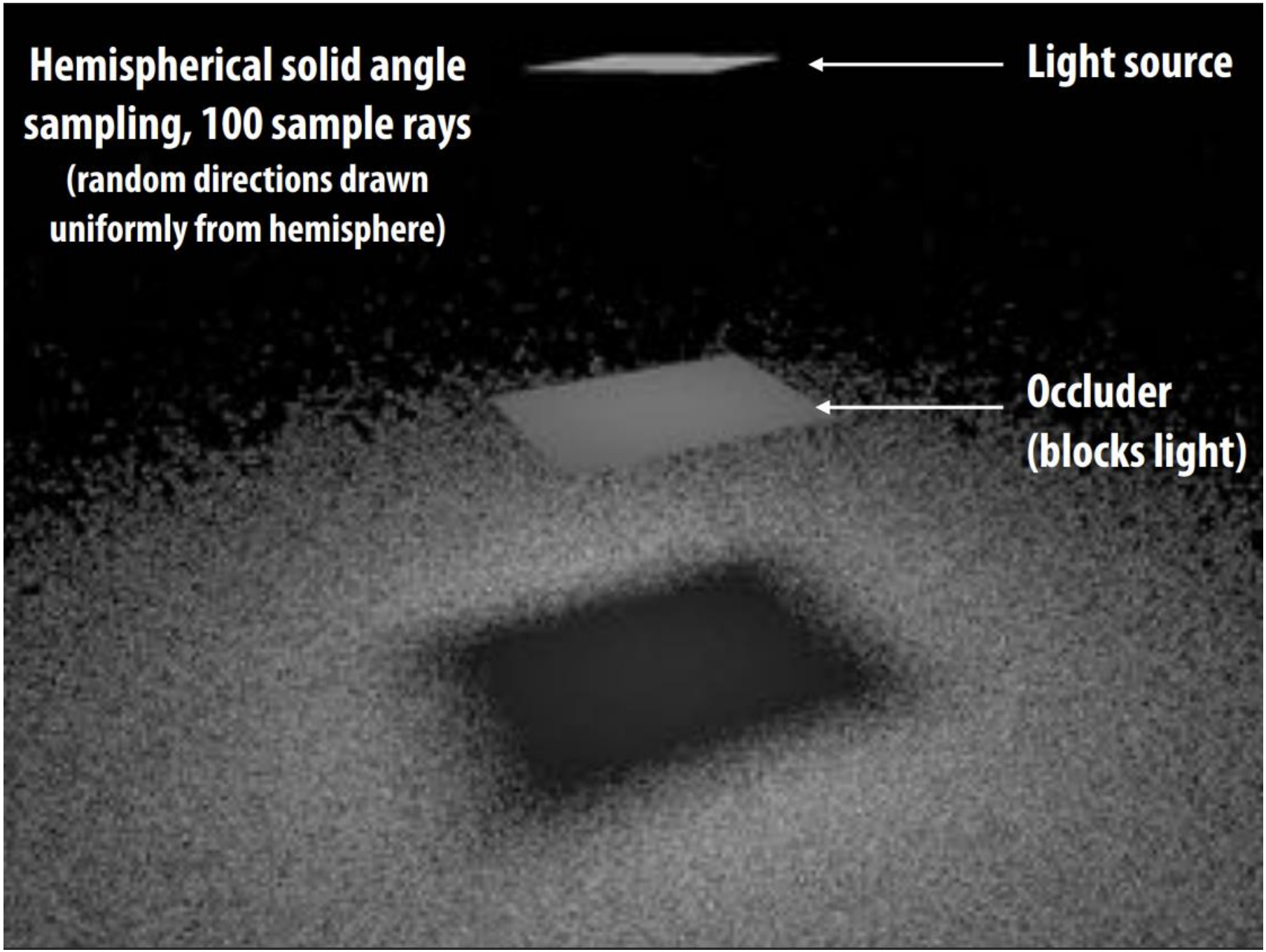
For each of N samples:

Generate random direction: ω_i

Compute incoming radiance arriving L_i **at p from direction:** ω_i

Compute incident irradiance due to ray: $dE_i = L_i cos\,\theta_i$

Accumulate $\frac{2\pi}{N} dE_i$ **into estimator**


Hemispherical solid angle sampling, 100 sample rays
(random directions drawn uniformly from hemisphere)
Light source
Occluder (blocks light)

Direct lighting: area integral

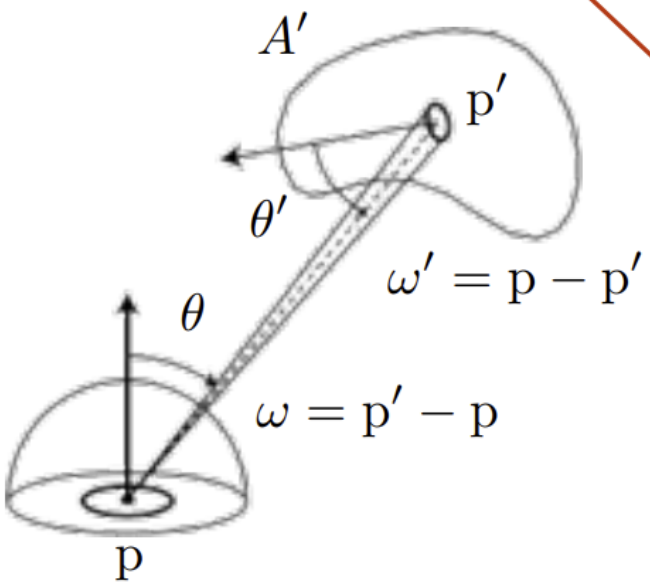
$$E(\mathrm{p}) = \int L(\mathrm{p}, \omega)\, \cos\theta \, \mathrm{d}\omega$$

Previously: just integrate over all directions

$$E(\mathrm{p}) = \int_{A'} L_o(\mathrm{p}', \omega')\, V(\mathrm{p}, \mathrm{p}')\, \frac{\cos\theta \cos\theta'}{|\mathrm{p} - \mathrm{p}'|^2}\, \mathrm{d}A'$$

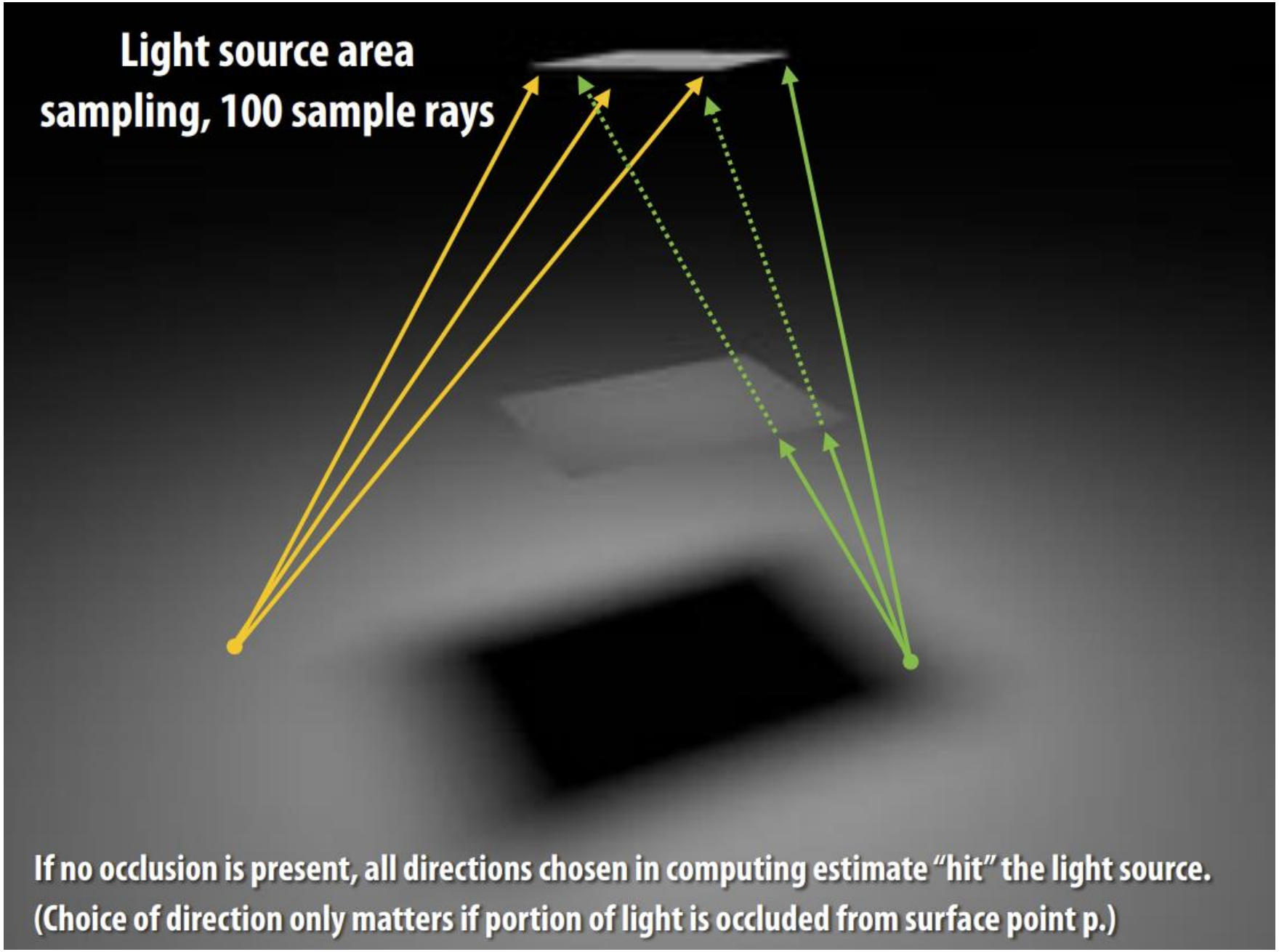
Change of variables to integrate over area of light

$$d\omega = \frac{dA}{|\mathrm{p}' - \mathrm{p}|^2} = \frac{dA' cos\,\theta'}{|\mathrm{p}' - \mathrm{p}|^2}$$



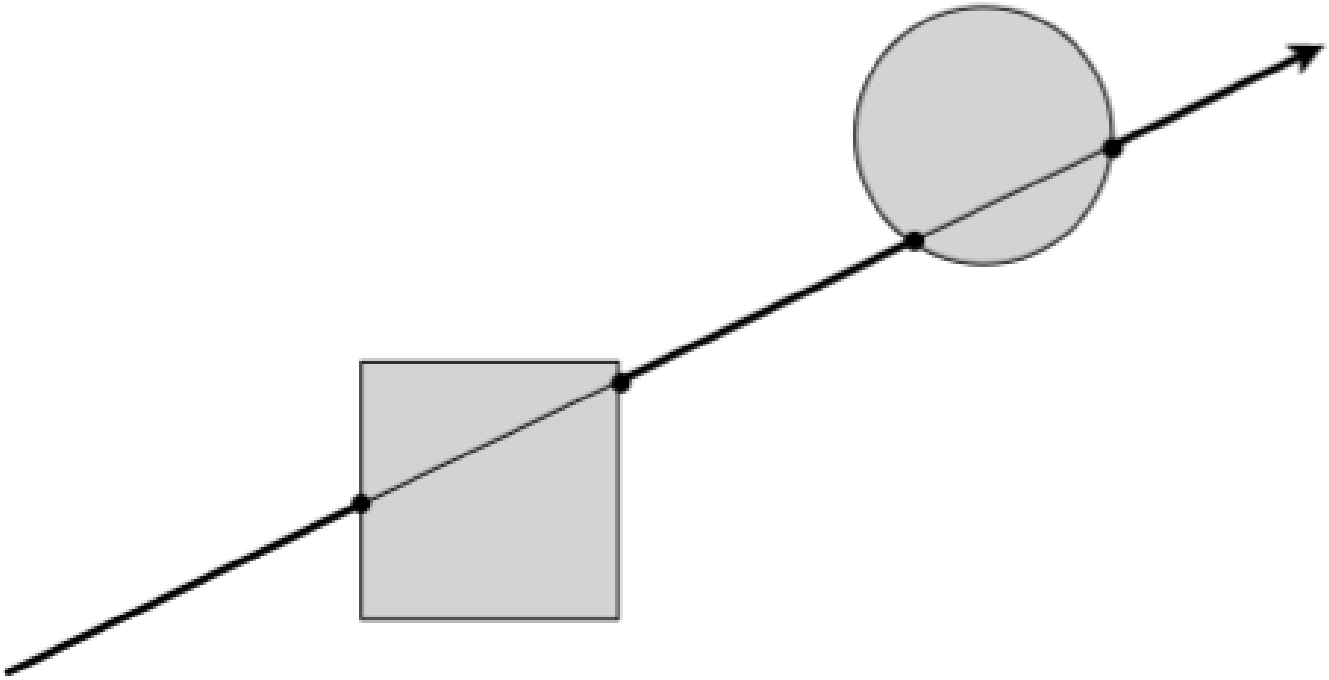
Binary visibility function:
1 if p′ is visible from p, 0 otherwise
(accounts for light occlusion)

Outgoing radiance from light point p, in direction w′ towards p


Light source area
sampling, 100 sample rays
If no occlusion is present, all directions chosen in computing estimate "hit" the light source.
(Choice of direction only matters if portion of light is occluded from surface point p.)

Intersection and shading

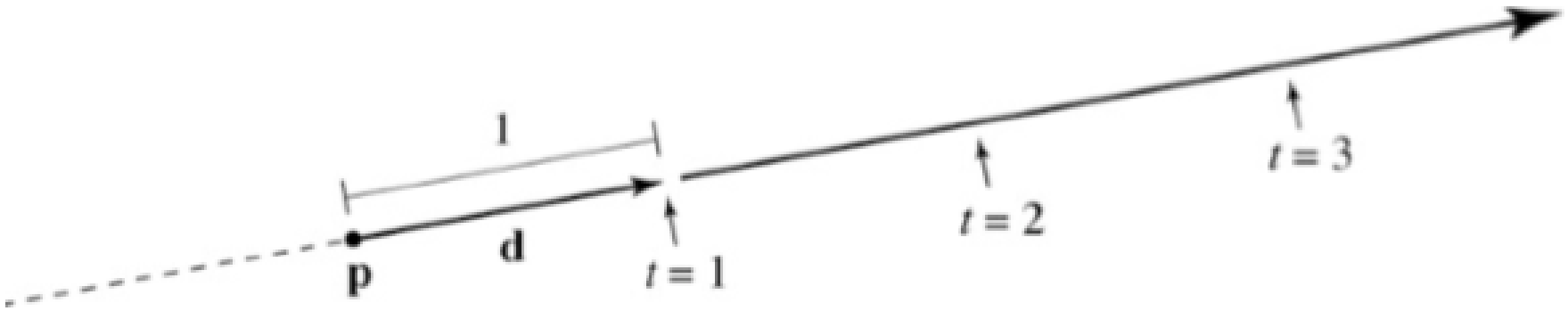
- Ray intersection



(Steve Marschner, MIT)

Ray

- Standard representation: $r(t) = p + td$ where p is the start point and d is the direction and the parameter t >0



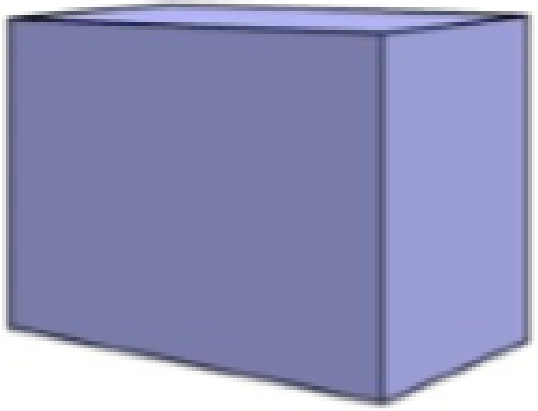
(Steve Marschner, MIT)

Ray-sphere intersection

- Solving algebra:
- $r(t) = p + td$
- $(x - x_0)^2 + (y - y_0)^2 + (z - z_0)^2 = R^2$
- Substitute:
- $(r_x(t) - x_0)^2 + (r_y(t) - y_0)^2 + (r_z(t) - z_0)^2 = R^2$
- Quadratic formula to solve *t*
  - There will be two *t* values

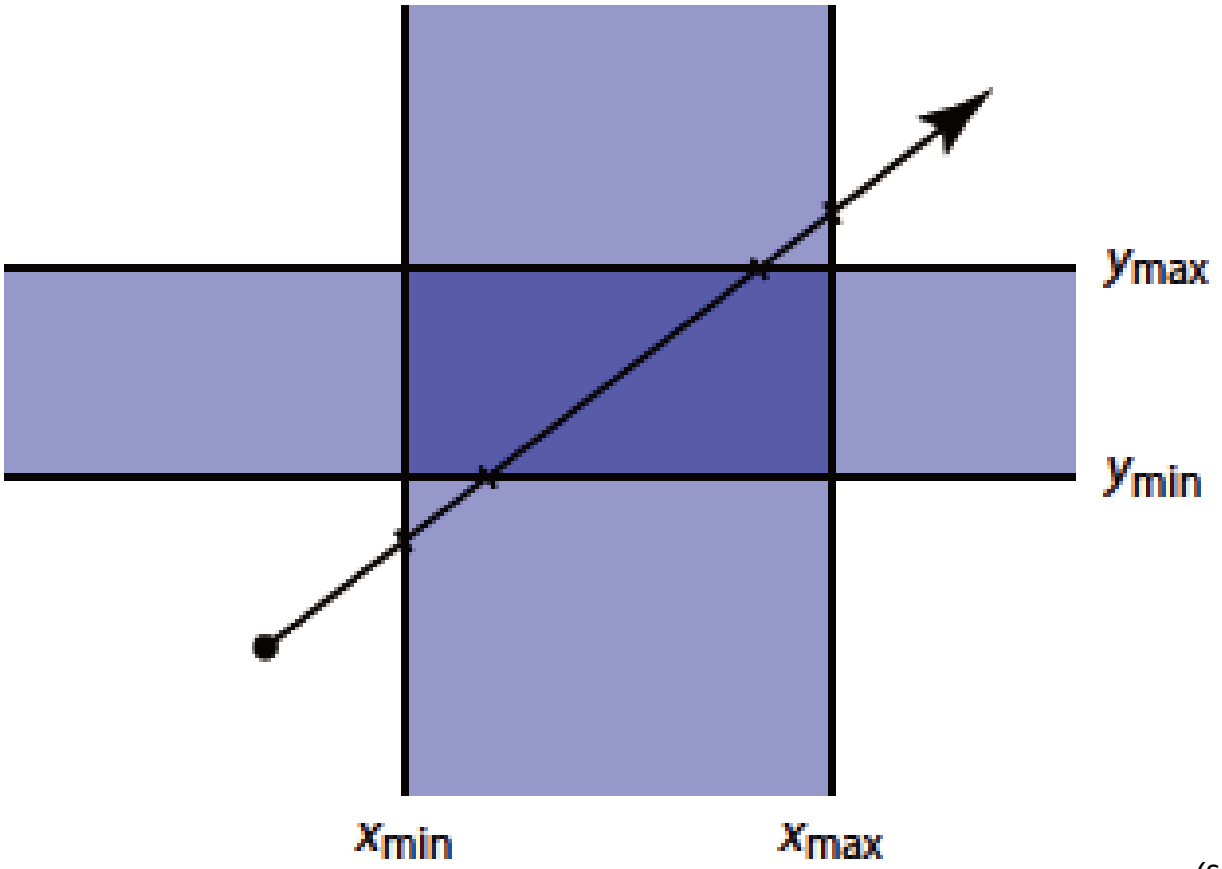
Ray-box intersection

- Intersect with 6 faces
- Again solving algebra
  - Ray equation
  - Plane equation
  - Boundary check
- Or intersect with boundaries



Ray-box intersection

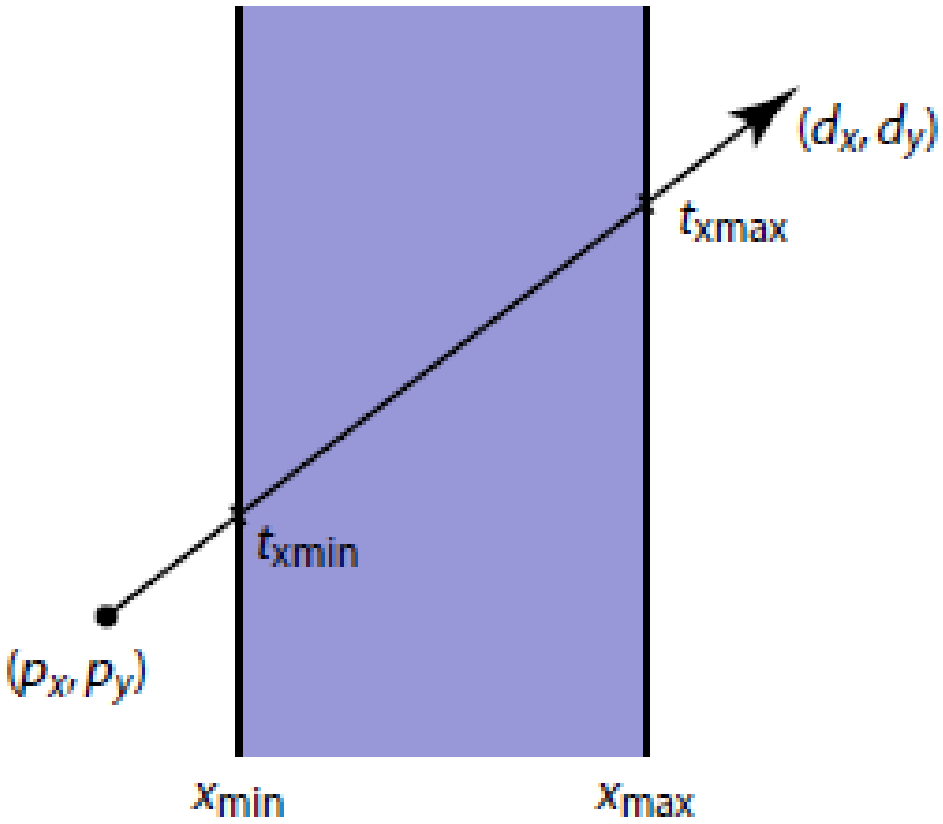
- Similar to clipping algorithm



(Steve Marschner, MIT)

Ray-box intersection

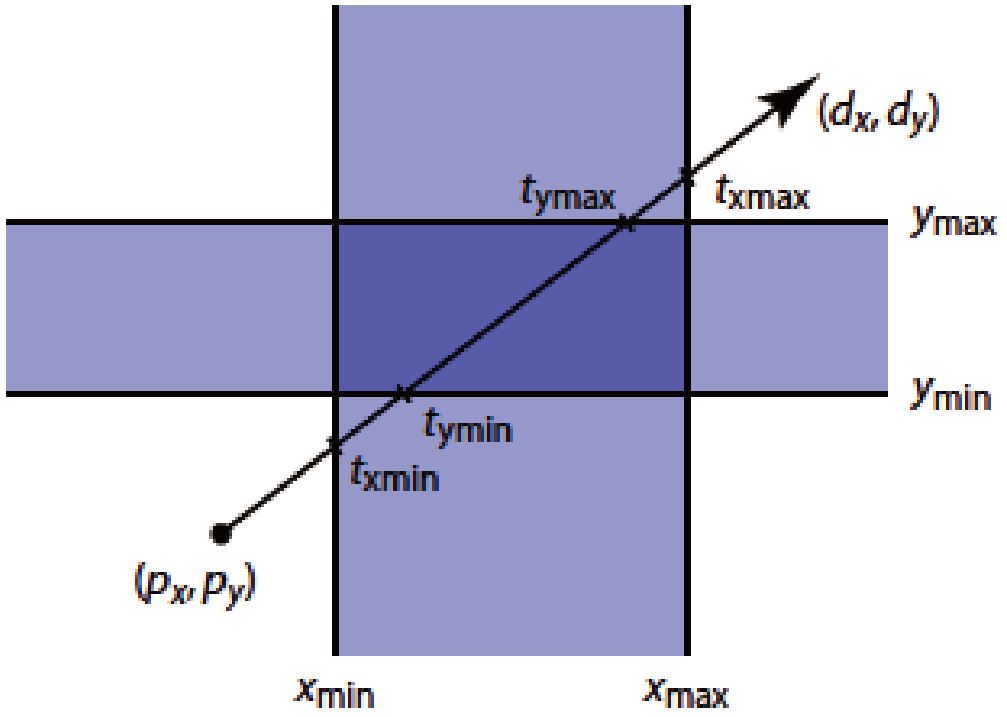
- Similar to clipping algorithm
- $p_x + t_{xmin} d_x = x_{min}$
- $p_x + t_{xmax} d_x = x_{max}$



(Steve Marschner, MIT)

Ray-box intersection

- Similar to clipping algorithm
- $p_x + t_{xmin} d_x = x_{min}$
- $p_x + t_{xmax} d_x = x_{max}$
- $p_y + t_{ymin} d_y = y_{min}$
- $p_y + t_{ymax} d_y = y_{max}$

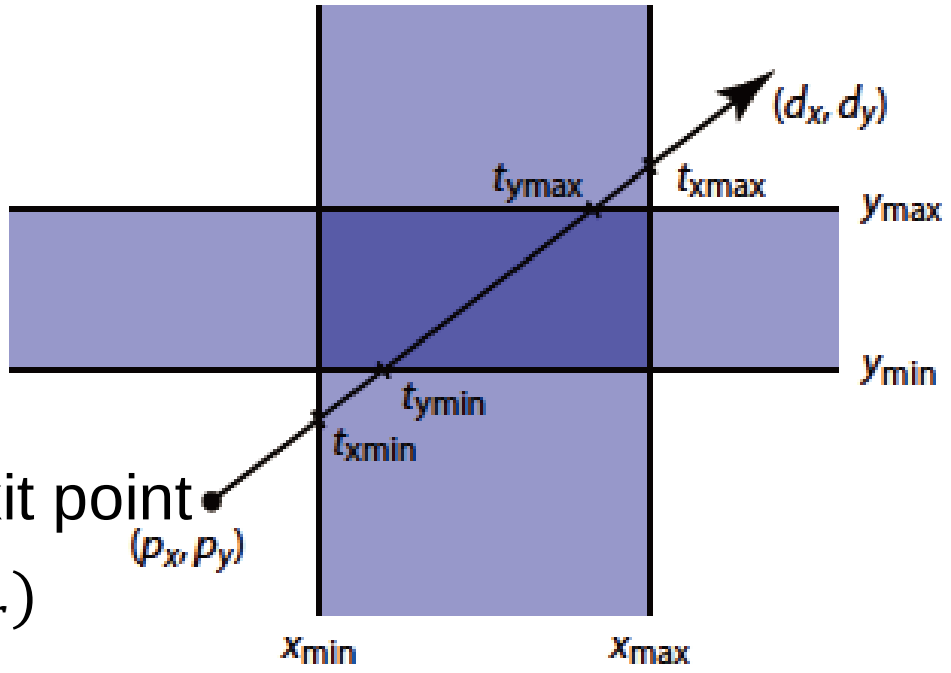


(Steve Marschner, MIT)

Ray-box intersection

- Find the enter point and exit point
- $t_{xenter} = \min(t_{xmin}, t_{xmax})$
- $t_{yenter} = \min(t_{ymin}, t_{ymax})$
- $t_{xexit} = max(t_{xmin}, t_{xmax})$
- $t_{yexit} = \max(t_{ymin}, t_{ymax})$

- Last enter point and first exit point
- $t_{enter} = \max(t_{xenter}, t_{yenter})$
- $t_{exit} = \min(t_{xexit}, t_{yexit})$



(Steve Marschner, MIT)

Real-time ray tracing

https://www.youtube.com/watch?v=aKqxonOrl4Q

Ray tracing acceleration

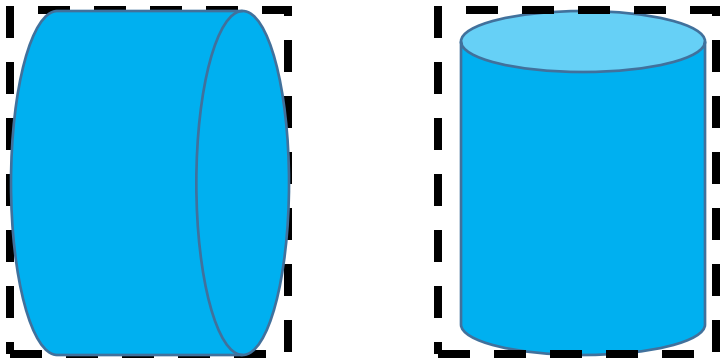
- Ray tracer spends most of the time in ray-surface intersection
- To improve
  - Make intersection more efficient
  - Only do intersection when necessary
  - Trace in parallel
  - …

Intersect when necessary

- Try to avoid meaningless intersections by using auxiliary data structures
  - BVH
  - Octree
  - BSPtree
  - Kdtree
  - Etc.

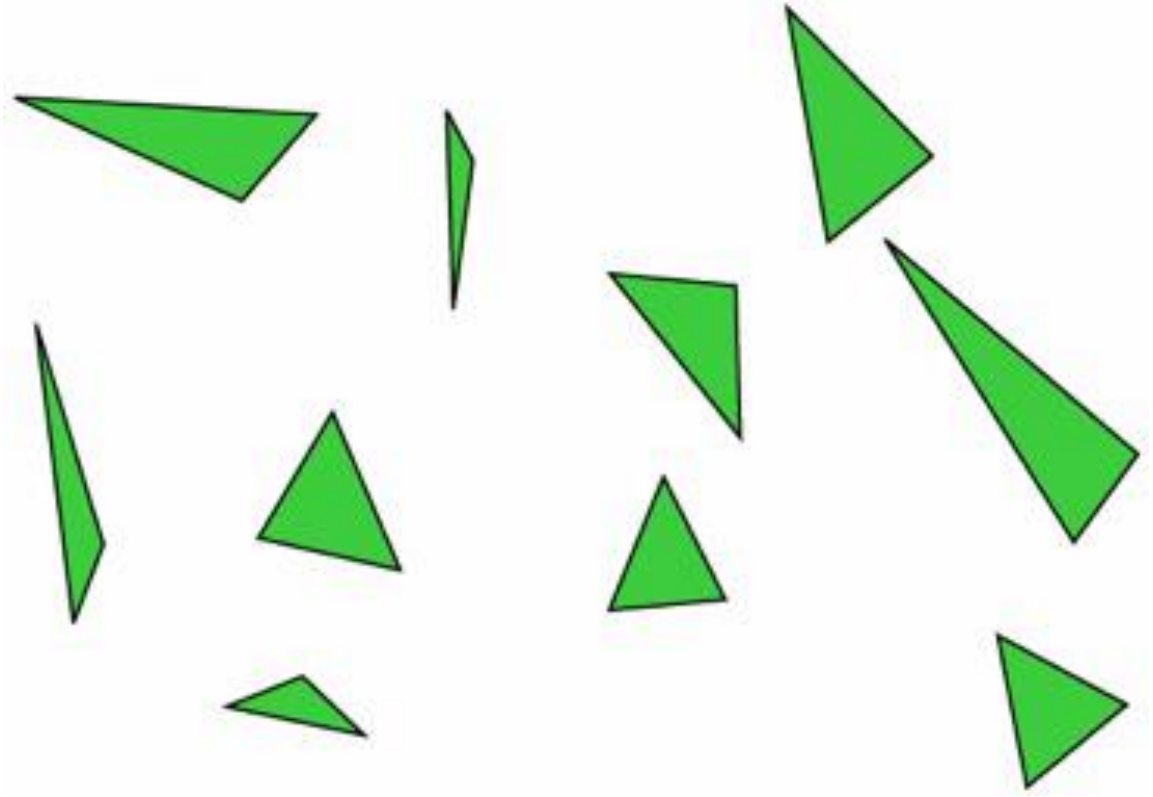
Bounding volume hierarchy (BVH)

- Object is contained within a volume
- First test the ray with the volume, then test the including objects when it hits the volume
- The volume should be simple (AABB boxes in most of time)
- Volume should fit the object tightly



Bounding volume hierarchy (BVH)

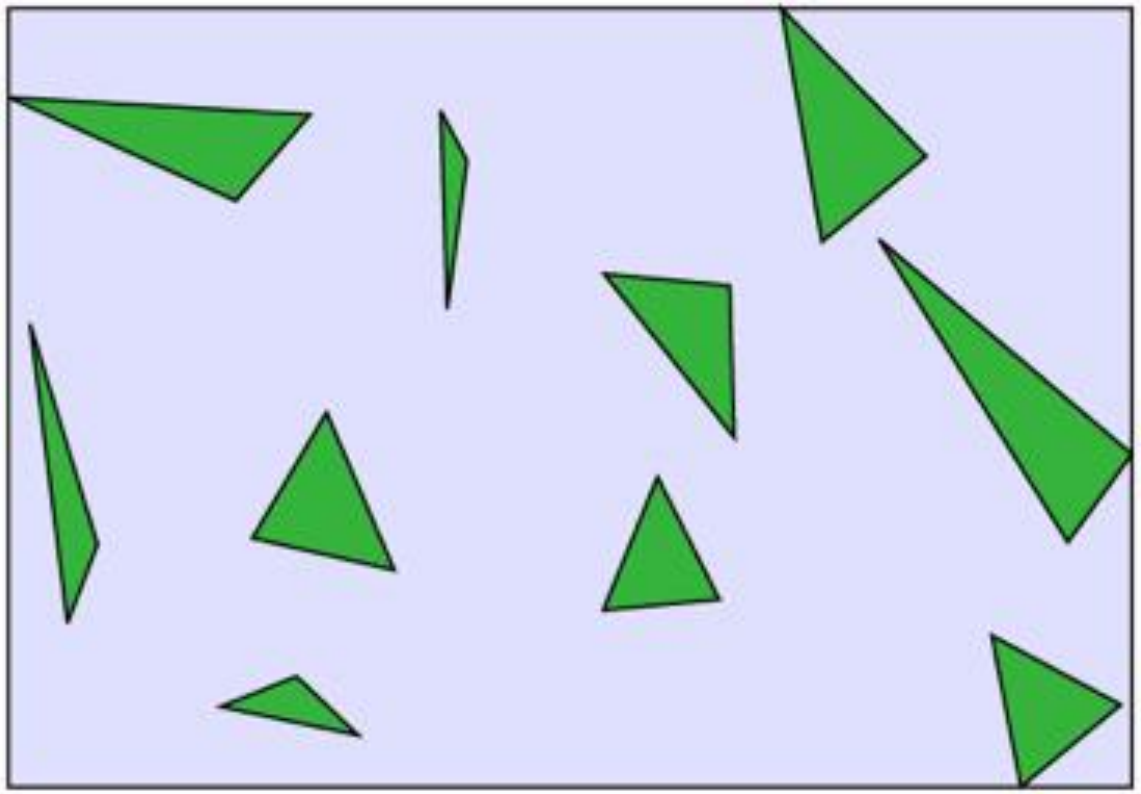
- Hierarchy of bounding volumes
- A tree



(Steve Marschner, MIT)

Bounding volume hierarchy (BVH)

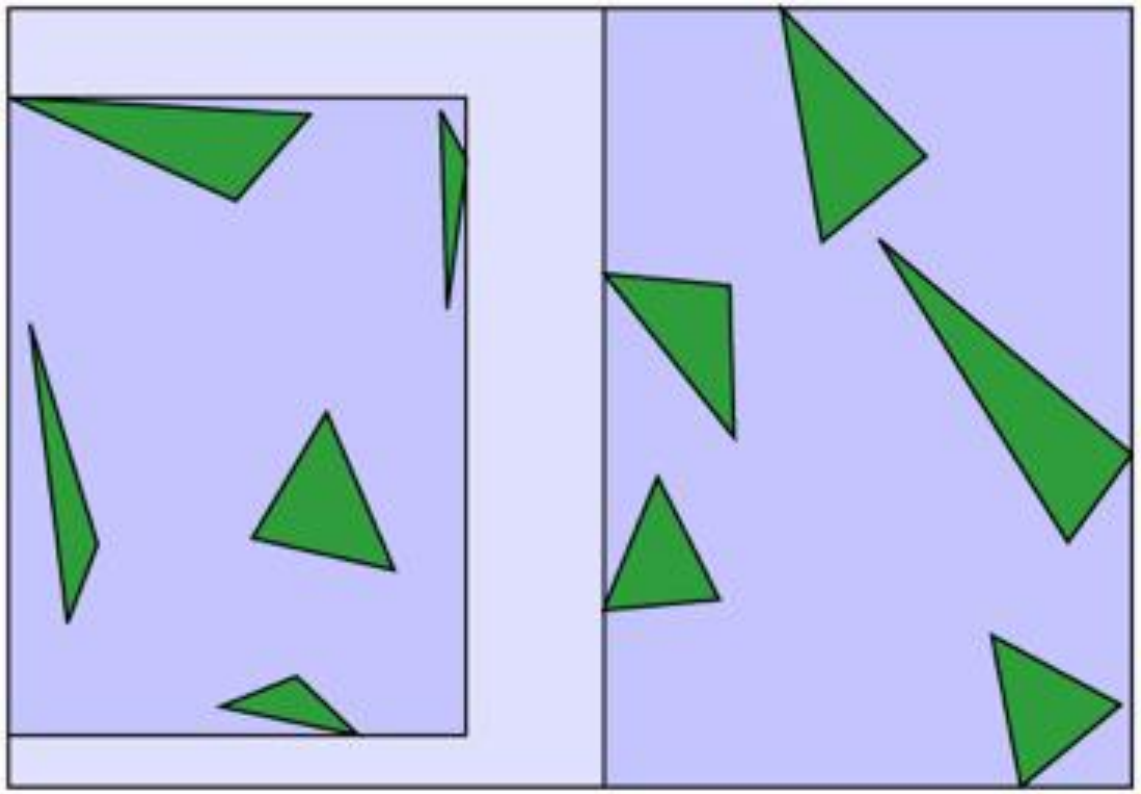
- Hierarchy of bounding volumes
- A tree



(Steve Marschner, MIT)

Bounding volume hierarchy (BVH)

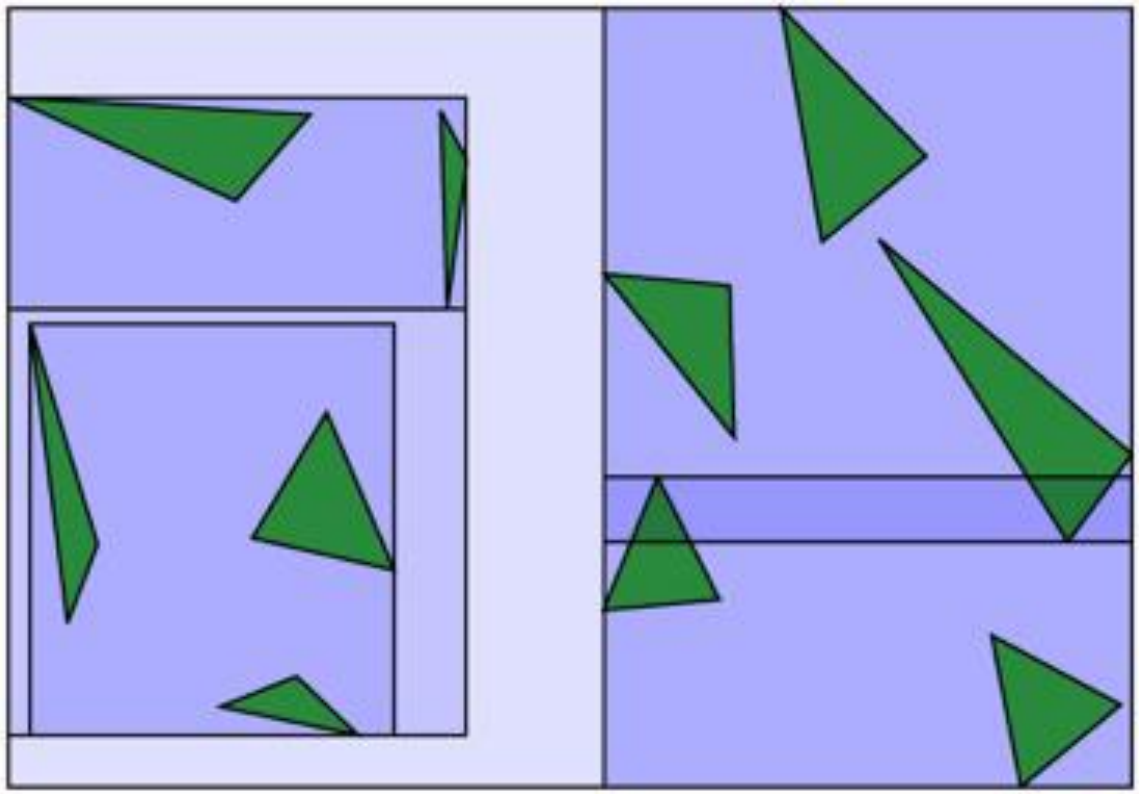
- Hierarchy of bounding volumes
- A tree



(Steve Marschner, MIT)

Bounding volume hierarchy (BVH)

- Hierarchy of bounding volumes
- A tree



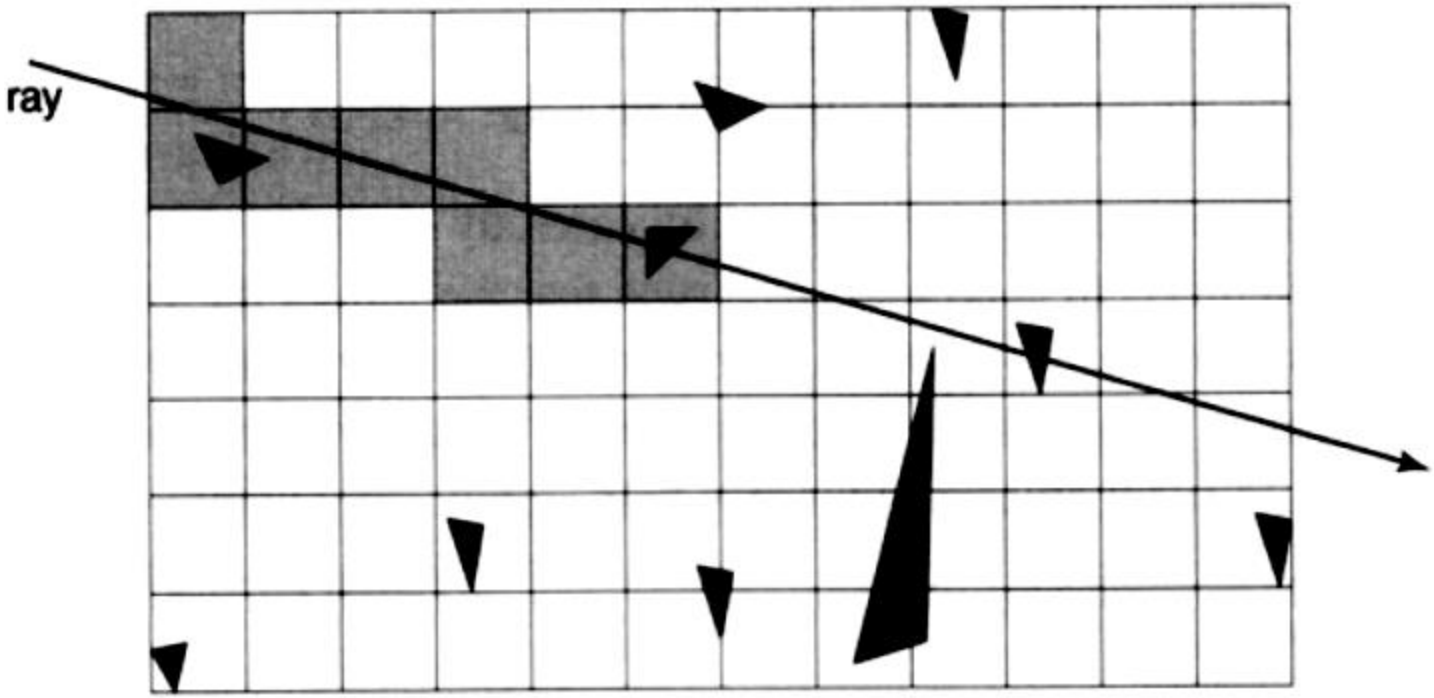
(Steve Marschner, MIT)

Bounding volume hierarchy (BVH)

- Building a BVH
- Partition smartly
- Balance the tree will generally improve the efficiency
- Expectation of the intersection cost

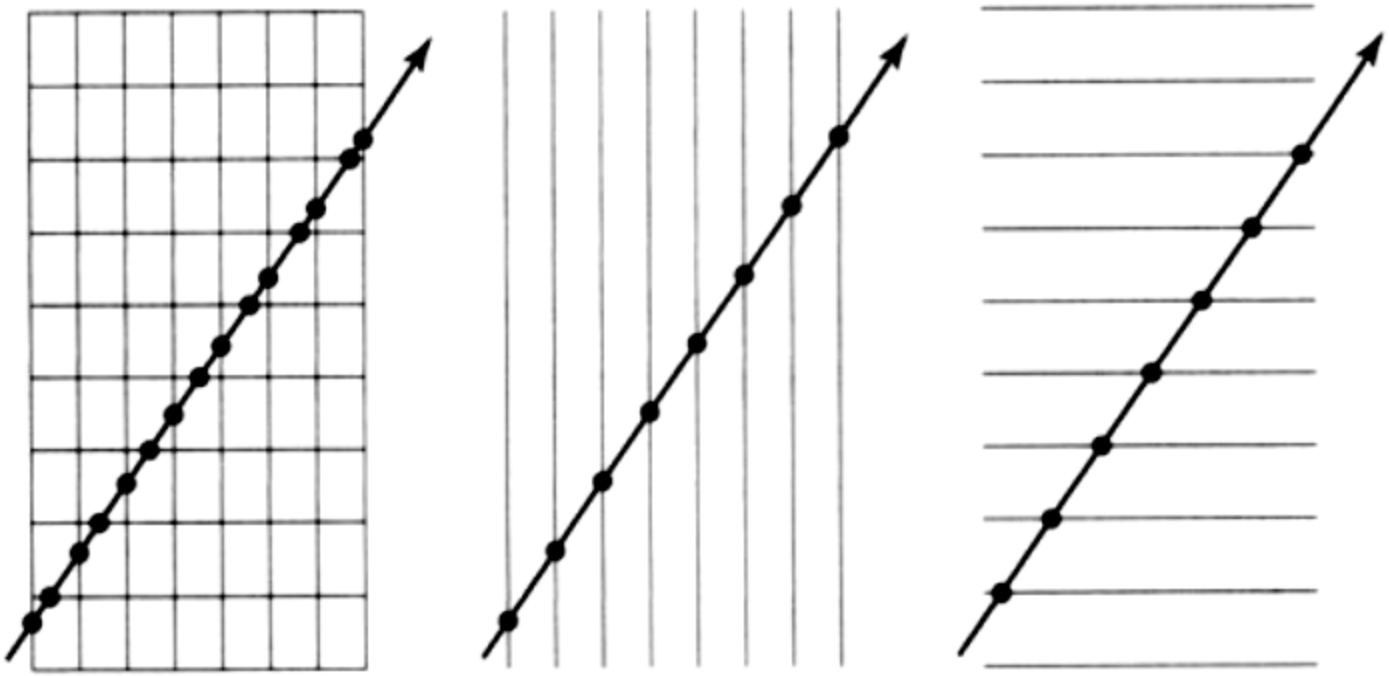
Octree

- Analogy to Grids in 3D
- Not group objects, but divide space, how?



Octree

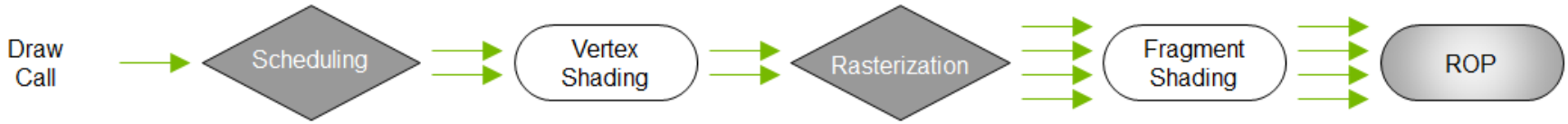
- Traverse the grid using enter point and exit point



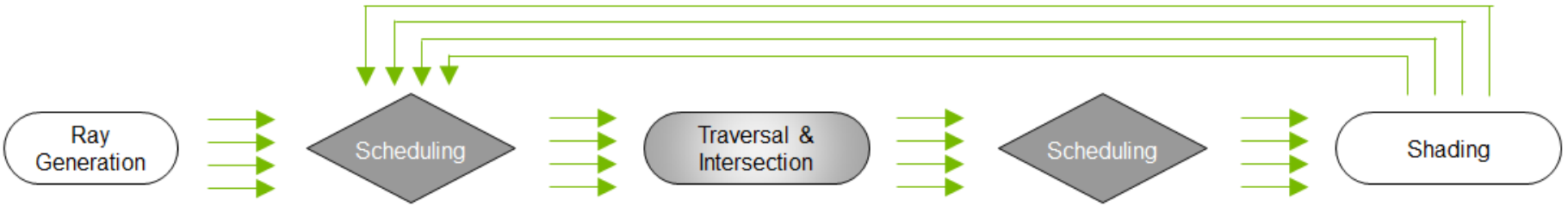
- Questions?

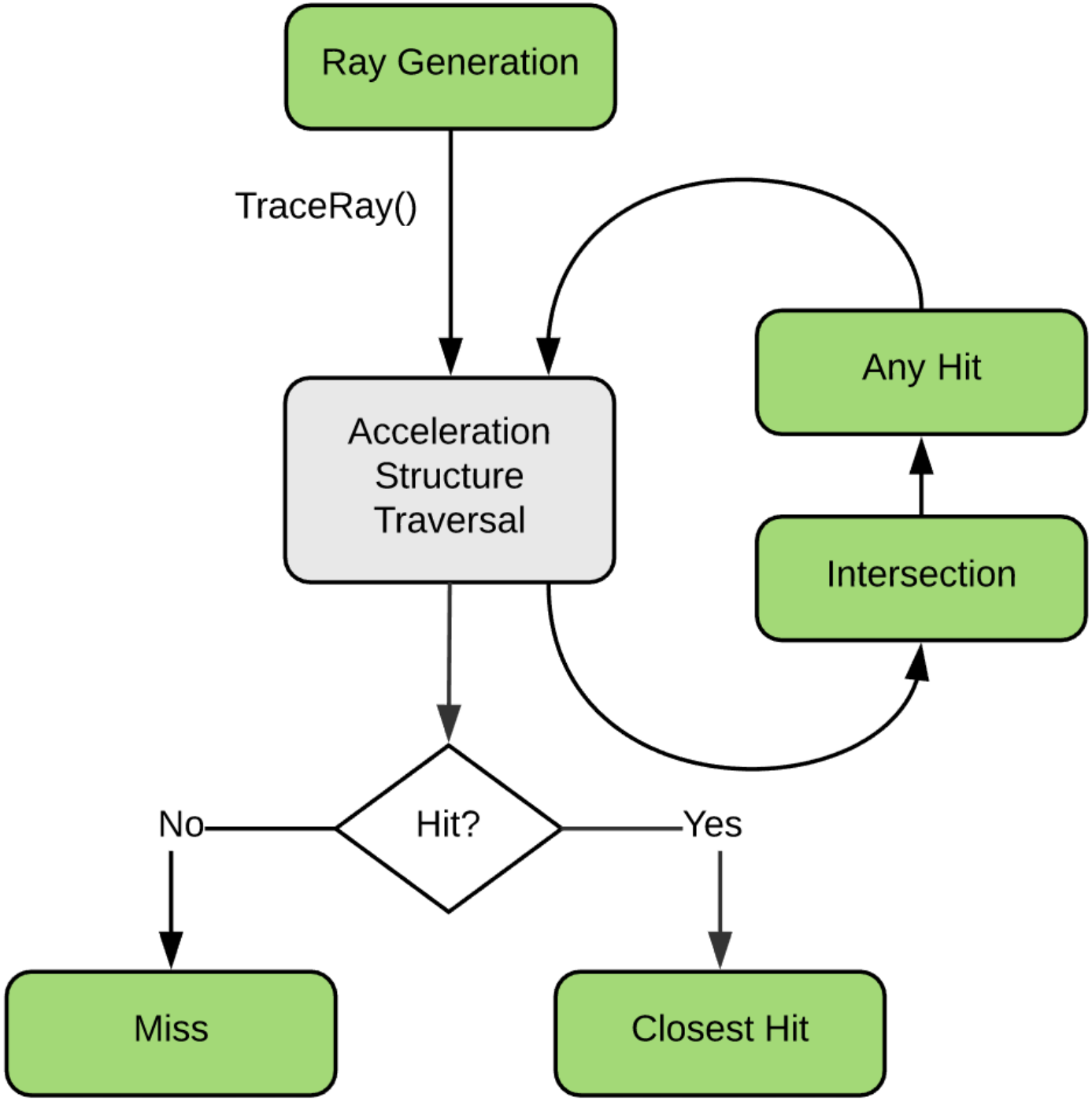
Raytracing Pipline

Rasterization



Ray Tracing




Ray Generation
TraceRay()
Acceleration Structure Traversal
Any Hit
Intersection
Hit?
No
Yes
Miss
Closest Hit

Some Ray Tracing APIs

- Hardware vendor specific:
  - OptiX, Embree, FireRays
- Cross-vendor APIs:
  - DirectX Raytracing, Vulkan RT
- Game engine APIs:
  - Unity, Unreal
- Different:
  - Audiences, learning curves, flexibility, performance, built-in optimizations

Today: Using DirectX For Sample Code

- Why?
  - DirectX widely used API for interactive graphics
  - Similar to Vulkan model
  - Abstracts some bits tricky for novices' ray tracers
  - Tutorial frameworks for easy experimentation

Five Types of Ray Tracing Shaders

- Ray tracing pipeline split into ***five*** shaders:
  - *A **ray generation shader** define how to start tracing rays*

Five Types of Ray Tracing Shaders

- Ray tracing pipeline split into ***five*** shaders:
  - *A **ray generation shader** define how to start tracing rays*
  - ***Intersection shader(s)** define how rays intersect geometry*

Five Types of Ray Tracing Shaders

- Ray tracing pipeline split into ***five*** shaders:
  - *A **ray generation shader*** *define how to start tracing rays*
  - ***Intersection shader(s)*** *define how rays intersect geometry*
  - ***Miss shader(s)*** *shading for when rays miss geometry*

Five Types of Ray Tracing Shaders

- Ray tracing pipeline split into ***five*** shaders:
  - *A **ray generation shader** define how to start tracing rays*
  - ***Intersection shader(s)** define how rays intersect geometry*
  - ***Miss shader(s)** shading for when rays miss geometry*
  - ***Closest-hit shader(s)** shading at the intersection point*

Five Types of Ray Tracing Shaders

- Ray tracing pipeline split into ***five*** shaders:
  - *A **ray generation shader*** *define how to start tracing rays*
  - ***Intersection shader(s)*** *define how rays intersect geometry*
  - ***Miss shader(s)*** *shading for when rays miss geometry*
  - ***Closest-hit shader(s)*** *shading at the closest intersection point*
  - ***Any-hit shader(s)*** *run once per hit** *(e.g., for transparency)*

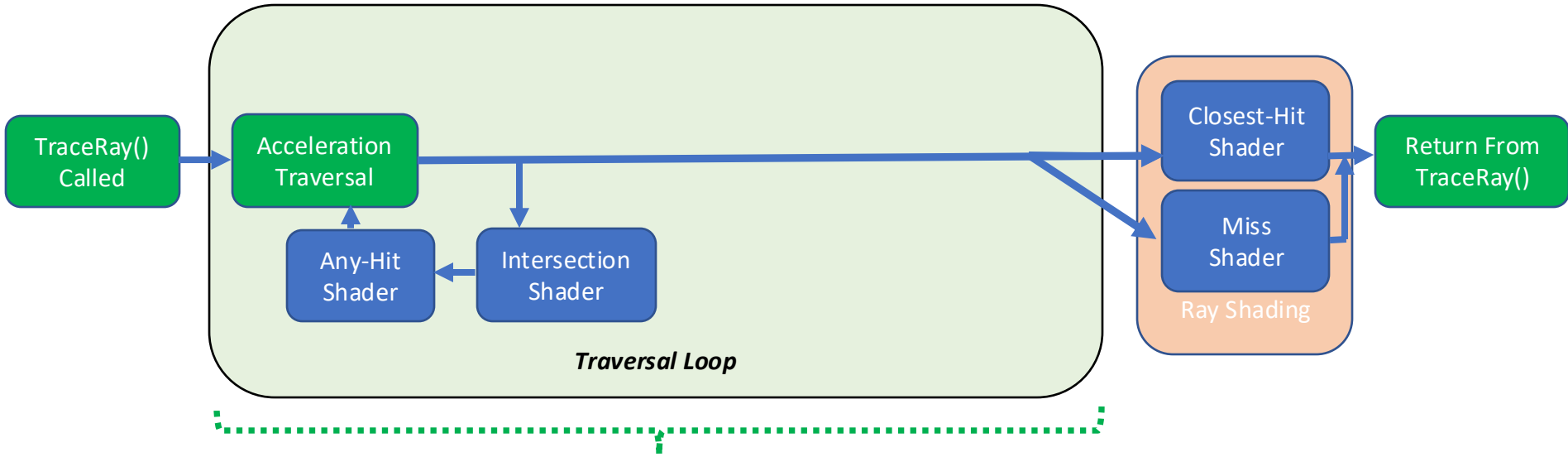
Five Types of Ray Tracing Shaders

- Ray tracing pipeline split into ***five*** shaders:
  - *A **ray generation shader*** ← Controls other shaders
  - ***Intersection shader(s)*** ← Defines object shapes (one shader per type)
  - ***Miss shader(s)***
  - ***Closest-hit shader(s)***
  - ***Any-hit shader(s)***

  ← Controls per-ray behavior (often many types)

How Do these Fit Together?

- Loop during ray tracing, testing hits until there's no more; then shade



Some important details here; learn later for advanced functionality

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work

```
[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Output image buffer
  - Communicates results with CPU

```
RWTexture<float4> gOutTex;
```

```
[shader("raygeneration")]
void MyRayGen()  {
    uint2  curPixel    = DispatchRaysIndex().xy;
    float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

    RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
    RayPayload payload = { float3(0, 0, 0) };

    TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

    outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Information about scene
  - Passed as input from the CPU

```
RWTexture<float4> gOutTex;
```

```
[shader("raygeneration")]
void MyRayGen()  {
    uint2  curPixel    = DispatchRaysIndex().xy;
    float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

    RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
    RayPayload payload = { float3(0, 0, 0) };

    TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

    outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Each ray returns some value
  - Return payload is user-defined
  - Often, like this one, just a color
- Before tracing, initialize payload

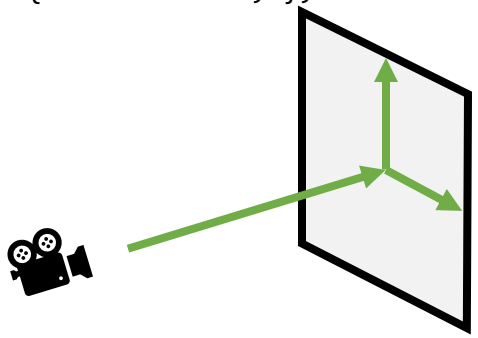
```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };
```

```
[shader("raygeneration")]
void MyRayGen()  {
    uint2  curPixel    = DispatchRaysIndex().xy;
    float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

    RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
    RayPayload payload = { float3(0, 0, 0) };

    TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

    outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- You write a function here
  - Computes per-pixel ray direction
  - Based on location on screen

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };
```

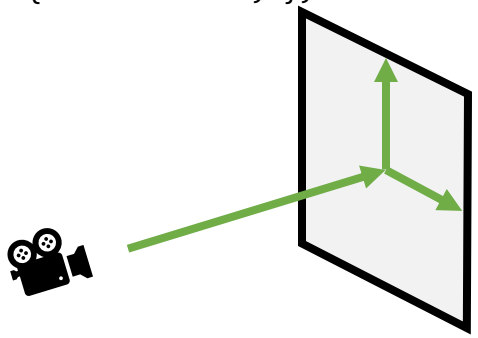


```
[shader("raygeneration")]
void MyRayGen()  {
    uint2  curPixel    = DispatchRaysIndex().xy;
    float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

    RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
    RayPayload payload = { float3(0, 0, 0) };

    TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

    outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- You write a function here
  - Computes per-pixel ray direction
  - Based on location on screen
- Setup the ray to trace

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };
```

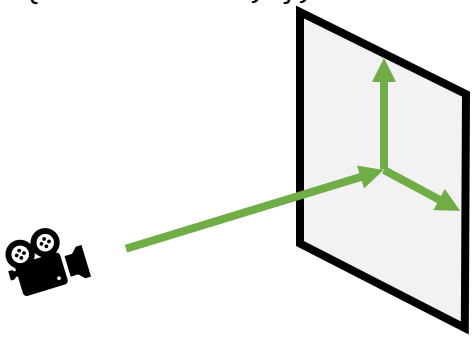


```
[shader("raygeneration")]
void MyRayGen()  {
    uint2  curPixel    = DispatchRaysIndex().xy;
    float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

    RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
    RayPayload payload = { float3(0, 0, 0) };

    TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

    outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- You write a function here
  - Computes per-pixel ray direction
  - Based on location on screen
- Setup the ray to trace
  - Min and max distances to search

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };
```



```
[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Trace your ray here

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };
```

```
[shader("raygeneration")]
void MyRayGen()  {
    uint2  curPixel    = DispatchRaysIndex().xy;
    float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

    RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
    RayPayload payload = { float3(0, 0, 0) };

    TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

    outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Trace your ray here
  - Scene BVH

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };
```

```
[shader("raygeneration")]
void MyRayGen()  {
    uint2  curPixel    = DispatchRaysIndex().xy;
    float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

    RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
    RayPayload payload = { float3(0, 0, 0) };

    TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

    outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Trace your ray here
  - Scene BVH
  - No special ray behaviors

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };
```

```
[shader("raygeneration")]
void MyRayGen()  {
    uint2  curPixel    = DispatchRaysIndex().xy;
    float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

    RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
    RayPayload payload = { float3(0, 0, 0) };

    TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

    outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Trace your ray here
  - Scene BVH
  - No special ray behaviors
  - What geometry should we test?
    - Bitmask; 0xFF → test all geometry

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };
```

```
[shader("raygeneration")]
void MyRayGen()  {
    uint2  curPixel    = DispatchRaysIndex().xy;
    float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

    RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
    RayPayload payload = { float3(0, 0, 0) };

    TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

    outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Trace your ray here
  - Scene BVH
  - No special ray behaviors
  - What geometry should we test?
    - Bitmask; 0xFF → test all geometry
  - Ray and payload from earlier

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };
```

```
[shader("raygeneration")]
void MyRayGen()  {
    uint2  curPixel    = DispatchRaysIndex().xy;
    float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

    RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
    RayPayload payload = { float3(0, 0, 0) };

    TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

    outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Which miss shader to use?
  - There's a list of miss shaders
  - Specify index of the one to use

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };

[shader("miss")]
void MyMiss(inout RayPayload payload)  {
     payload.color = float3( 0, 0, 1 );
}
```

```
[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Which miss shader to use?
  - There's a list of miss shaders
  - Specify index of the one to use
- In my tutorials, `MyMiss` is index 0
  - Why? First miss shader I loaded

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };

[shader("miss")]
void MyMiss(inout RayPayload payload)  {
     payload.color = float3( 0, 0, 1 );
}
```

```
[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Which ***hit group*** to use?
  - May have 1 *any-hit shader*
  - May have 1 *closest-hit shader*
  - May have 1 *intersection shader*

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };

[shader("miss")]
void MyMiss(inout RayPayload payload)  {
     payload.color = float3( 0, 0, 1 );
}

[shader("closesthit")]
void MyClosestHit(inout RayPayload data,
                  BuiltinTriangleIntersectAttribs
attribs)  {
     data.color = float3( 1, 0, 0 );
}

[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- Remember:
  - Ray generation shader starts work
- Which ***hit group*** to use?
  - May have 1 *any-hit shader*
  - May have 1 *closest-hit shader*
  - May have 1 *intersection shader*
- Here, has just one shader
  - It's index 0 → specified first on load

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };

[shader("miss")]
void MyMiss(inout RayPayload payload)  {
     payload.color = float3( 0, 0, 1 );
}

[shader("closesthit")]
void MyClosestHit(inout RayPayload data,
                  BuiltinTriangleIntersectAttribs
attribs)  {
     data.color = float3( 1, 0, 0 );
}

[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- How to read at high level:
  - For each pixel determine ray

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };

[shader("miss")]
void MyMiss(inout RayPayload payload)  {
     payload.color = float3( 0, 0, 1 );
}

[shader("closesthit")]
void MyClosestHit(inout RayPayload data,
                  BuiltinTriangleIntersectAttribs
attribs)  {
     data.color = float3( 1, 0, 0 );
}

[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- How to read at high level:
  - For each pixel determine ray
  - Shoot the ray

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };

[shader("miss")]
void MyMiss(inout RayPayload payload)  {
     payload.color = float3( 0, 0, 1 );
}

[shader("closesthit")]
void MyClosestHit(inout RayPayload data,
                  BuiltinTriangleIntersectAttribs
attribs)  {
     data.color = float3( 1, 0, 0 );
}

[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- How to read at high level:
  - For each pixel determine ray
  - Shoot the ray
  - If it misses? Return blue

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };

[shader("miss")]
void MyMiss(inout RayPayload payload)  {
     payload.color = float3( 0, 0, 1 );
}

[shader("closesthit")]
void MyClosestHit(inout RayPayload data,
                  BuiltinTriangleIntersectAttribs
attribs)  {
     data.color = float3( 1, 0, 0 );
}

[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- How to read at high level:
  - For each pixel determine ray
  - Shoot the ray
  - If it misses? Return blue
  - If it hits? Return red

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };

[shader("miss")]
void MyMiss(inout RayPayload payload)  {
     payload.color = float3( 0, 0, 1 );
}
```

```
[shader("closesthit")]
void MyClosestHit(inout RayPayload data,
                  BuiltinTriangleIntersectAttribs
attribs)  {
     data.color = float3( 1, 0, 0 );
}
```

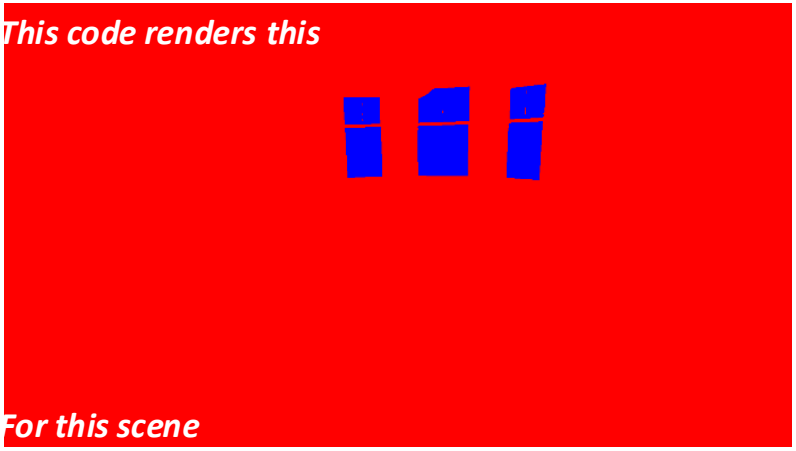
```
[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer

- How to read at high level:
  - For each pixel determine ray
  - Shoot the ray
  - If it misses? Return blue
  - If it hits? Return red
  - Output our result

```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };

[shader("miss")]
void MyMiss(inout RayPayload payload)  {
     payload.color = float3( 0, 0, 1 );
}

[shader("closesthit")]
void MyClosestHit(inout RayPayload data,
                  BuiltinTriangleIntersectAttribs
attribs)  {
     data.color = float3( 1, 0, 0 );
}

[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

Really SIMPLE GPU Ray Tracer



```
RWTexture<float4> gOutTex;
struct RayPayload { float3 color; };

[shader("miss")]
void MyMiss(inout RayPayload payload)  {
     payload.color = float3( 0, 0, 1 );
}

[shader("closesthit")]
void MyClosestHit(inout RayPayload data,
                  BuiltinTriangleIntersectAttribs
attribs)  {
     data.color = float3( 1, 0, 0 );
}

[shader("raygeneration")]
void MyRayGen()  {
     uint2  curPixel    = DispatchRaysIndex().xy;
     float3 pixelRayDir = normalize(
getRayDirFromPixelID( curPixel ) );

     RayDesc ray        = { gCamera.posW, 0.0f,
pixelRayDir, 1e+38f };
     RayPayload payload = { float3(0, 0, 0) };

     TraceRay( gRtScene, RAY_FLAG_NONE, 0xFF, 0, 1, 0,
ray, payload );

     outTex[curPixel]   = float4( payload.color, 1.0f );
}
```

What about a Real Example?

- Examples from my DXR tutors: http://intro-to-dxr.cwyman.org
  - Click on “code walkthrough”
  - Not quite equivalent to any of those, but close

References

- SIGGRAPH 2019 courses
  - Introduction to Real-time Ray Tracing
- Steve Marschner, CS4620/5620 Computer Graphics, Cornell
- Ed Angel, CS/EECE 433 Computer Graphics, University of New Mexico
- Elif Tosun, Computer Graphics, The University of New York

- Questions?